Implementation of Simulated Annealing to Solve 8-Queens problem

```
import random

def get_h(state):
    h = 0
    n = len(state)
    for i in range(n):
        for j in range(i + 1, n):
            if state[i] == state[j] or abs(state[i] - state[j]) == abs(i - j):
                h += 1
    return h

def get_best_neighbor(state):
    n = len(state)
    best_state = list(state)
    best_h = get_h(state)

    for col in range(n):
        for row in range(n):
            if row != state[col]:
                neighbor = list(state)
                neighbor[col] = row
                h = get_h(neighbor)
                if h < best_h:
                    best_h = h
                    best_state = neighbor
```

```
    return best_state, best_h

def print_board(state):
    n = len(state)
    for r in range(n):
        print(" ".join("Q" if state[c] == r else "." for c in range(n)))
    print()

def hill_climbing(state):
    current = list(state)
    h = get_h(current)
    step = 0

    while True:
        print(f"Step {step} | State: {current} | Cost: {h}")
        print_board(current)

        if h == 0:
            print("Solved!")
            return current

        neighbor, n_h = get_best_neighbor(current)

        if n_h < h:
            current = neighbor
            h = n_h
            step += 1
        else:
```

```
        print("Restarting...")
        current = [random.randint(0, len(state) - 1) for _ in range(len(state))]
        h = get_h(current)
        step += 1

# Run
start = [0, 0, 0, 0]
hill_climbing(start)
```

```
Step 0 | State: [0, 0, 0, 0] | Cost: 6
Q Q Q Q
. . . .
. . . .
. . . .

Step 1 | State: [0, 3, 0, 0] | Cost: 3
Q . Q Q
. . . .
. . . .
. Q . .

Step 2 | State: [1, 3, 0, 0] | Cost: 1
. . Q Q
Q . . .
. . . .
. Q . .

Step 3 | State: [1, 3, 0, 2] | Cost: 0
. . Q .
Q . . .
. . . Q
. Q . .

Solved!
[1]: [1, 3, 0, 2]
```

```
[ ]:
```